

Card Collections – Promise To Pay Service

Architecture Design Document

Version

1.0

Author

Pitabas Pradhan

Date

June 2026

1. Executive Summary

1.1 Purpose

This document describes the architecture, design decisions, implementation approach, and operational considerations for the Card Collections Promise-To-Pay (PTP) Service.

The solution was developed as a lightweight, production-inspired backend service that enables collection agents to create and manage delinquent credit card collection cases and record customer Promise-To-Pay commitments.

The implementation demonstrates modern software engineering practices including:

- Clean Architecture
- Domain Driven Design (DDD)
- RESTful API Design
- Idempotent Request Processing
- Audit Trail Management
- Dependency Injection
- Domain Events
- Global Exception Handling
- Unit Testing

The architecture intentionally separates business rules from infrastructure concerns, allowing the system to evolve from an in-memory implementation to a production-grade distributed service with minimal changes.

1.2 Business Objective

Financial institutions manage millions of delinquent accounts every year.

When a customer misses payments and becomes delinquent, collection teams engage with the customer and negotiate repayment commitments.

One common outcome is a Promise-To-Pay (PTP), where the customer agrees to make a payment on a specific future date.

The business requires:

- Creation of collection cases
- Recording of Promise-To-Pay commitments
- Tracking of case status changes
- Prevention of duplicate requests
- Auditable business operations
- Future integration with enterprise event-driven platforms

This service addresses those requirements.

1.3 Scope

In Scope

- Collection Case creation
- Collection Case retrieval
- Promise-To-Pay creation
- Status transitions
- Idempotency handling
- Audit trail recording
- Domain event publishing
- Validation
- Exception handling
- Unit testing

Out of Scope

- Authentication
- Authorization
- SQL Server persistence
- Distributed caching
- Message broker integration
- Customer communication channels
- Payment processing
- Reporting

These capabilities are identified as future enhancements.

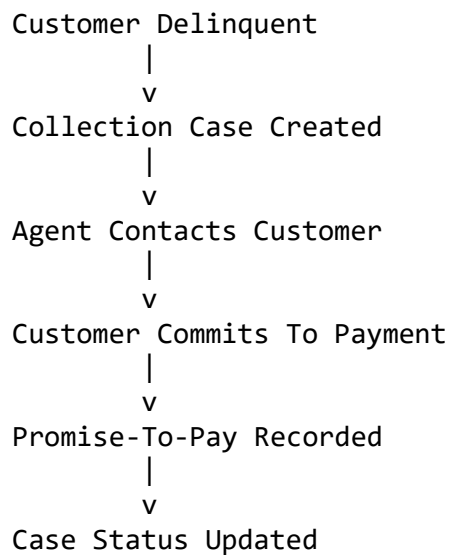
2. Business Context

2.1 Current Business Process

The collections lifecycle begins when a customer becomes delinquent on a credit card account.

The collections department creates a case to track interactions and repayment commitments.

A typical process is:



2.2 Key Stakeholders

Collections Agent

Responsible for:

- Creating customer commitments
- Tracking repayment promises
- Managing collection cases

Collections Manager

Responsible for:

- Operational oversight
- Audit review
- Compliance monitoring

Operations Team

Responsible for:

- Application support
- Incident management
- Production monitoring

Technology Team

Responsible for:

- Development
 - Deployment
 - Maintenance
 - Enhancements
-

2.3 Business Challenges

Financial systems frequently encounter:

Duplicate Requests

Network interruptions may cause users to submit the same request multiple times.

Without protection:

Request #1

|

v

Promise Created

Request #2

|

v

Duplicate Promise Created

This creates data integrity issues.

Lack of Traceability

Business actions must be traceable for:

- Compliance
- Operational support
- Customer disputes
- Audit reviews

Tight Coupling

Traditional systems often combine:

- Business Logic
- Persistence Logic
- API Logic

inside a single layer.

This increases maintenance costs and reduces agility.

3. Problem Statement

The organization requires a backend service capable of:

1. Creating collection cases.
2. Recording Promise-To-Pay commitments.
3. Maintaining audit history.

4. Preventing duplicate submissions.
5. Supporting future enterprise integrations.
6. Maintaining clear separation of concerns.

The solution should demonstrate enterprise-grade design principles while remaining simple enough to support rapid development and testing.

4. Functional Requirements

FR-01 Create Collection Case

The system shall create a new collection case.

Inputs:

- CustomerId
- MaskedCardNumber
- DelinquentAmount

Outputs:

- CaseId
- Status

Default status:

Open

FR-02 Retrieve Collection Case

The system shall return collection case information.

Inputs:

- CaseId

Outputs:

- Customer information
 - Delinquent amount
 - Current status
-

FR-03 Create Promise-To-Pay

The system shall record a Promise-To-Pay commitment.

Inputs:

- Caseld
- Amount
- PromiseDate
- AgentId

Outputs:

- PromiseToPayId
 - Updated status
-

FR-04 Status Transition

When a Promise-To-Pay is created:

Open
|
v
PromiseToPay

The case status shall automatically transition.

FR-05 Audit Trail

The system shall record all Promise-To-Pay activities in an audit log.

FR-06 Idempotent Processing

The system shall reject duplicate Promise-To-Pay requests using an Idempotency Key.

FR-07 Domain Event Publication

The system shall publish a domain event whenever a Promise-To-Pay is successfully created.

5. Non-Functional Requirements

NFR-01 Maintainability

The solution shall follow Clean Architecture principles.

Benefits:

- Separation of concerns
 - Independent testing
 - Easier enhancements
-

NFR-02 Testability

Business logic shall be independently testable.

The solution shall support unit testing without requiring API execution.

NFR-03 Scalability

The architecture shall support migration to:

- SQL Server
- Redis
- Azure Service Bus
- Kafka

without significant redesign.

NFR-04 Reliability

The system shall support idempotent processing to prevent duplicate business operations.

NFR-05 Extensibility

Future business capabilities should be added with minimal impact on existing code.

Examples:

- Installment arrangements

- Payment plans
 - Settlement offers
 - Customer notifications
-

6. Assumptions and Constraints

Assumptions

- Single-node deployment.
 - In-memory storage acceptable for assessment.
 - Trusted internal users.
 - Low transaction volume.
-

Constraints

- Limited implementation timeframe.
 - No external database.
 - No external messaging platform.
 - No authentication provider.
-

7. Architectural Drivers

The following drivers influenced the architecture:

Business Drivers

- Data integrity
- Auditability
- Future growth

Technical Drivers

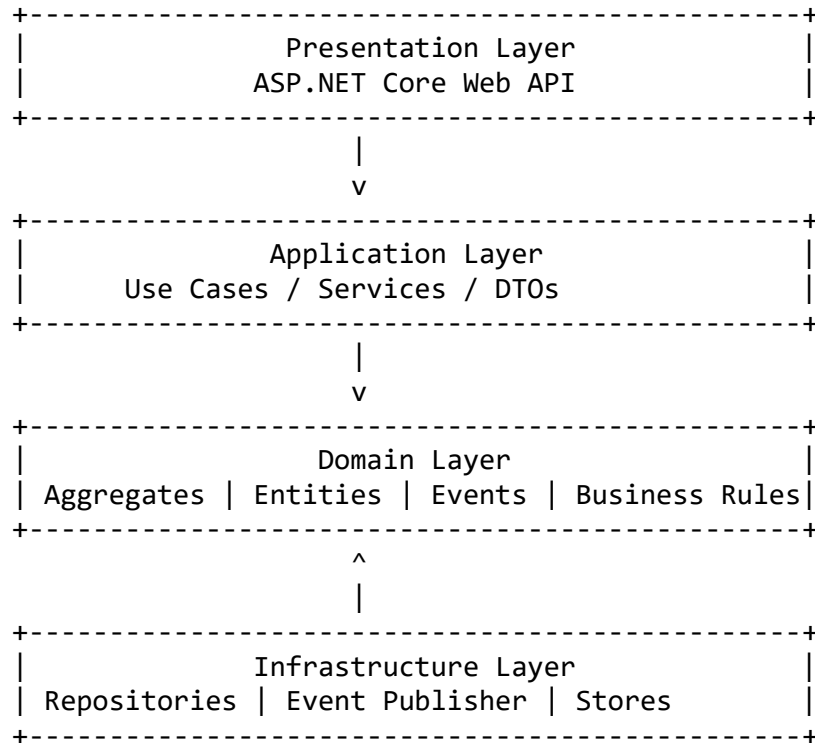
- Maintainability
- Testability
- Separation of concerns

Operational Drivers

- Simplicity
- Reliability
- Traceability

These drivers directly influenced the decision to adopt Clean Architecture and Domain Driven Design.

8. High-Level Architecture



The architecture ensures that business rules remain independent from frameworks, databases, and external technologies.

9. Clean Architecture Implementation

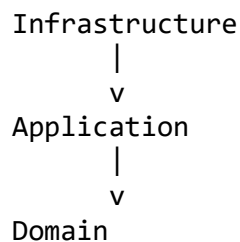
9.1 Architectural Approach

The solution follows the Clean Architecture pattern popularized by Robert C. Martin (Uncle Bob).

The primary goal of Clean Architecture is to ensure that business rules remain independent of external concerns such as:

- Frameworks
- Databases
- Messaging Systems
- User Interfaces
- Infrastructure Components

The architecture enforces dependency direction toward the domain.



The Domain layer has no dependencies on any other project.

9.2 Benefits

Separation of Concerns

Each layer has a clearly defined responsibility.

Testability

Business rules can be tested independently without APIs or databases.

Maintainability

Changes in infrastructure do not affect domain logic.

Extensibility

New implementations can be added without modifying business rules.

Examples:

- SQL Server Repository
 - MongoDB Repository
 - Redis Idempotency Store
 - Azure Service Bus Publisher
-

10. Layer Responsibilities

10.1 API Layer

Project:

`CardCollections.Api`

Responsibilities:

- HTTP Endpoint Exposure
- Request Handling
- Response Generation
- Dependency Injection Configuration
- Middleware Registration

Contains:

Controllers

Middleware

`Program.cs`

The API layer contains no business logic.

10.2 Application Layer

Project:

`CardCollections.Application`

Responsibilities:

- Use Cases
- Application Services
- DTOs
- Contracts
- Orchestration

Contains:

Services
Interfaces
DTOs

Examples:

CollectionCaseService
PromiseToPayService

The Application layer coordinates activities but does not contain core business rules.

10.3 Domain Layer

Project:

CardCollections.Domain

Responsibilities:

- Business Rules
- Entities
- Aggregate Root
- Domain Events

Contains:

Entities
Enums
Events

Examples:

CollectionCase
PromiseToPay
AuditEntry

The Domain layer represents the heart of the solution.

10.4 Infrastructure Layer

Project:

CardCollections.Infrastructure

Responsibilities:

- Persistence
- Event Publishing
- Idempotency Storage
- External Integrations

Contains:

Repositories
Event Publishers
Stores

Examples:

InMemoryCollectionCaseRepository
InMemoryIdempotencyStore
FakeEventPublisher

11. Domain Driven Design (DDD)

11.1 Why DDD?

The collections domain contains business concepts that must remain protected from technical concerns.

Examples:

Collection Case

Promise-To-Pay

Audit Trail

Case Status

DDD provides:

- Rich Domain Model
- Explicit Business Concepts
- Strong Consistency Boundaries

11.2 Ubiquitous Language

The solution uses business terminology directly in code.

Examples:

CollectionCase

PromiseToPay

CaseStatus

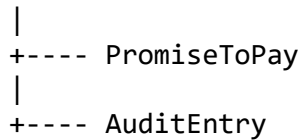
AuditEntry

This reduces translation between business and technical teams.

12. Domain Model

12.1 Domain Overview

CollectionCase



CollectionCase acts as the Aggregate Root.

12.2 CollectionCase

Represents:

A delinquent customer collection case.

Attributes:

Id

CustomerId

MaskedCardNumber

DelinquentAmount

Status

Responsibilities:

Maintain consistency

Manage promises

Maintain audit history

Control state transitions

12.3 PromiseToPay

Represents:

A customer commitment to pay.

Attributes:

PromiseToPayId

Amount

PromiseDate

CreatedBy

Characteristics:

Immutable after creation

Owned by CollectionCase

12.4 AuditEntry

Represents:

Business activity history

Attributes:

AuditEntryId

Action

UserId

Timestamp

Purpose:

Compliance

Traceability

Operational Support

12.5 CaseStatus

Current states:

Open

PromiseToPay

Future states may include:

Resolved

BrokenPromise

Closed

Settlement

13. Aggregate Design

13.1 Aggregate Root

CollectionCase is the Aggregate Root.

CollectionCase

```
|  
+-- PromiseToPay  
|  
+-- AuditEntry
```

External consumers interact only through CollectionCase.

13.2 Why Aggregate Root?

Without an Aggregate Root:

PromiseToPay
|
+ Direct Updates

could bypass business rules.

With Aggregate Root:

CollectionCase
|
+ Business Validation
|
+ State Transition
|
+ Audit Creation

all modifications remain consistent.

13.3 Invariants

The Aggregate enforces:

Rule 1

Promise-To-Pay amount must be greater than zero.

Rule 2

Promise date cannot be in the past.

Rule 3

Status transitions occur only through business methods.

Rule 4

Audit entries are created when PTP is created.

14. Request Processing Flow

Create Collection Case

Client
|
v
CasesController

```
|  
v  
CollectionCaseService  
|  
v  
CollectionCase  
|  
v  
Repository  
|  
v  
Response
```

Create Promise-To-Pay

```
Client  
|  
v  
CasesController  
|  
v  
PromiseToPayService  
|  
v  
Idempotency Validation  
|  
v  
CollectionCase  
|  
v  
Create PromiseToPay  
|  
v  
Create AuditEntry  
|  
v  
Publish Domain Event  
|  
v  
Repository  
|  
v  
Response
```

15. API Design

Design Principles

The API follows RESTful conventions.

Characteristics:

Resource Oriented

Stateless

Predictable URLs

JSON Payloads

Endpoint Design

Create Case

POST /api/cases

Creates a collection case.

Retrieve Case

GET /api/cases/{id}

Returns case information.

Create Promise-To-Pay

POST /api/cases/{id}/promise-to-pay

Creates a customer payment commitment.

16. DTO Design

DTOs isolate external contracts from domain entities.

Benefits:

Loose Coupling

Versioning

Security

Flexibility

Examples:

CreateCaseRequest

CaseResponse

CreatePromiseToPayRequest

PromiseToPayResponse

The API never exposes domain entities directly.

17. State Transition Model

The current workflow supports:

```
Open
|
v
PromiseToPay
```

Transition is triggered by:

CreatePromiseToPay()

Future workflow may evolve into:

```
Open
|
v
PromiseToPay
|
+----- Fulfilled
|
+----- BrokenPromise
|
+----- Settlement
```

The aggregate root controls all transitions to guarantee consistency.

18. Idempotency Design

18.1 Overview

One of the most important requirements of the solution is the prevention of duplicate Promise-To-Pay (PTP) creation.

In distributed systems, duplicate requests can occur due to:

- Network timeouts
- Browser retries
- User double-clicks
- Reverse proxy retries
- API Gateway retries

Without idempotency protection, the same business operation could be processed multiple times.

Example:

Request 1
|
v
Create PTP

Request 2
|
v
Create PTP Again

Result:

Duplicate Promise-To-Pay Records

18.2 Idempotency Strategy

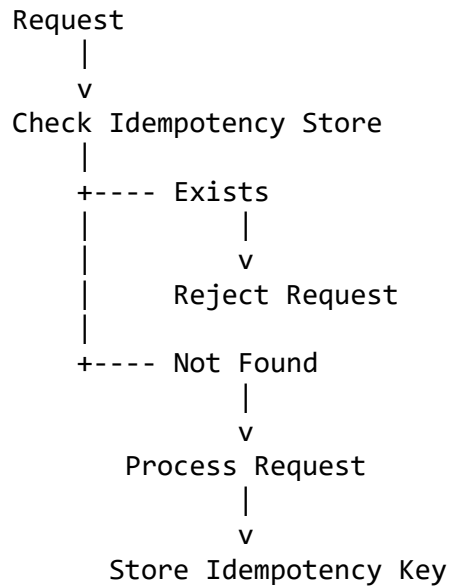
The service uses an Idempotency Key supplied by the caller.

Example:

POST /api/cases/{id}/promise-to-pay

Idempotency-Key: ptp-001

Processing Flow:



18.3 Implementation

Current implementation:

```
IIIdempotencyStore
|
v
InMemoryIdempotencyStore
```

The store maintains processed keys.

Example:

ptp-001

ptp-002

ptp-003

Duplicate requests are rejected.

18.4 Future Enhancement

Production implementation would store:

IdempotencyKey

RequestHash

ResponsePayload

CreatedTimestamp

Potential technologies:

Redis

SQL Server

Cosmos DB

19. Audit Trail Design

19.1 Purpose

Financial systems require traceability.

Every business operation should be auditable.

Audit records support:

- Compliance
 - Regulatory Reviews
 - Operational Support
 - Customer Investigations
 - Production Troubleshooting
-

19.2 Current Design

Audit information is represented using:

AuditEntry

Structure:

```
AuditEntry
|
+-- Id
+-- Action
+-- UserId
+-- Timestamp
```

19.3 Audit Flow

When a Promise-To-Pay is created:

```
Create PromiseToPay
      |
      v
Create AuditEntry
```

Both actions occur within the same aggregate operation.

19.4 Example Audit Record

```
{
  "action": "PromiseToPayCreated",
  "userId": "AGENT001",
  "timestamp": "2026-06-03T10:30:00Z"
}
```

19.5 Future Enhancements

Production audit records may include:

CorrelationId

Client IP

Device Information

Request Payload

Old Value

New Value

20. Domain Event Design

20.1 Overview

Domain Events represent significant business occurrences.

Examples:

PromiseToPayCreated

CaseClosed

SettlementAccepted

Domain Events help decouple business operations from downstream consumers.

20.2 Event Published

Current implementation publishes:

PromiseToPayCreatedEvent

when a Promise-To-Pay is successfully created.

20.3 Event Flow

```
graph TD; A[Create Promise-To-Pay] --> B[PromiseToPayCreatedEvent]; B --> C[Event Publisher];
```

20.4 Benefits

Loose Coupling

Business operations remain independent from downstream systems.

Extensibility

New subscribers can be added without changing domain logic.

Scalability

Supports future event-driven architecture.

20.5 Current Implementation

IEventPublisher

|

v

FakeEventPublisher

The publisher currently logs or simulates publication.

20.6 Future Integration

Potential integrations:

Azure Service Bus

Kafka

RabbitMQ

AWS SNS/SQS

21. Validation Strategy

21.1 Overview

Validation protects business integrity.

The solution validates requests before business processing.

21.2 Validation Location

Validation is implemented in the Application Layer.

CollectionCaseService

PromiseToPayService

Reason:

Business Use Cases

|

v

Application Layer

Controllers remain thin.

21.3 Collection Case Validation

CustomerId

Required.

CustomerId cannot be empty.

MaskedCardNumber

Required.

MaskedCardNumber cannot be empty.

DelinquentAmount

Must be positive.

Amount > 0

21.4 Promise-To-Pay Validation

Amount

Must be positive.

Amount > 0

PromiseDate

Cannot be in the past.

PromiseDate >= Today

Collection Case

Must exist.

Case must be found before creating PTP.

Idempotency

Duplicate keys are rejected.

22. Exception Handling Strategy

22.1 Overview

Unhandled exceptions create poor client experiences.

The solution implements centralized exception handling.

22.2 Middleware

Implementation:

ExceptionMiddleware

Responsibilities:

Catch Exceptions

Generate Standard Responses

Prevent Stack Trace Exposure

22.3 Flow

```
graph TD
    Request --> v1[ ]
    v1 --> ExceptionMiddleware
    ExceptionMiddleware --> v2[ ]
    v2 --> Controller
    Controller --> v3[ ]
    v3 --> Service
```

If an exception occurs:

Service
|
v
Exception
|
v
Middleware
|
v
JSON Response

22.4 Validation Errors

Example:

```
{  
  "error": "CustomerId is required."  
}
```

HTTP Status:

400 Bad Request

22.5 Unexpected Errors

Example:

```
{  
  "error": "An unexpected error occurred."  
}
```

HTTP Status:

500 Internal Server Error

23. Dependency Injection Strategy

23.1 Overview

Dependency Injection (DI) provides loose coupling and testability.

The solution uses the built-in ASP.NET Core DI container.

23.2 Service Registrations

Repositories:

ICollectionCaseRepository

Implementation:

InMemoryCollectionCaseRepository

Idempotency:

IIIdempotencyStore

Implementation:

InMemoryIdempotencyStore

Events:

IEventPublisher

Implementation:

FakeEventPublisher

Services:

ICollectionCaseService

IPromiseToPayService

23.3 Lifetime Selection

Repository:

Singleton

Reason:

In-Memory Data Store

Idempotency Store:

Singleton

Reason:

Shared Request History

Application Services:

Scoped

Reason:

One Instance Per Request

24. Testing Strategy

24.1 Objectives

Testing verifies:

- Business Rules
 - State Transitions
 - Validation
 - Idempotency
 - Application Logic
-

24.2 Unit Testing Framework

Framework:

MSTest

24.3 Domain Tests

CreatePromiseToPay_ShouldChangeStatusToPromiseToPay

Verifies:

Open

|

v

PromiseToPay

transition.

CreatePromiseToPay_ShouldAddPromiseToPay

Verifies:

PromiseToPay Collection Updated

CreatePromiseToPay_ShouldCreateAuditEntry

Verifies:

Audit Entry Generated

24.4 Application Tests

CreateCase_ShouldReturnCaseResponse

Verifies:

Case Creation Workflow

CreateCase_ShouldThrowForInvalidAmount

Verifies:

Validation Rules

24.5 Promise-To-Pay Tests

CreatePromiseToPay_ShouldCreatePromiseToPay

Verifies:

PTP Creation

CreatePromiseToPay_ShouldRejectDuplicateIdempotencyKey

Verifies:

Idempotency Logic

CreatePromiseToPay_ShouldThrowWhenCaseNotFound

Verifies:

Error Handling

24.6 End-to-End Testing

Manual Swagger testing validates:

Create Case

Get Case

Create Promise-To-Pay

Duplicate Promise-To-Pay

These tests verify integration across all architectural layers.

25. Security Considerations

25.1 Overview

Although security implementation is outside the scope of the assessment, the architecture has been designed to support enterprise-grade security controls.

The solution follows the principle of designing for security even when security features are not implemented.

25.2 Authentication

Current State:

Not Implemented

Production Recommendation:

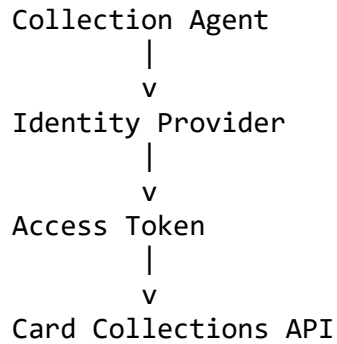
Azure AD

OAuth 2.0

OpenID Connect

JWT Authentication

Example:



25.3 Authorization

Future implementation should support role-based access control.

Example Roles:

Collection Agent

Collection Manager

Operations Support

Administrator

Example Permissions:

Create Collection Case

Create Promise-To-Pay

View Cases

View Audit History

25.4 Data Protection

Sensitive customer information should be protected.

Recommended controls:

TLS 1.2+

Encryption At Rest

Secrets Management

Data Masking

25.5 API Security

Recommended protections:

Rate Limiting

API Gateway

Web Application Firewall

Request Validation

Threat Detection

26. Scalability Considerations

26.1 Current State

The assessment implementation uses:

In-Memory Repository

In-Memory Idempotency Store

Suitable for:

Development

Testing

Demonstration

26.2 Production Scaling Strategy

Persistence Layer

Current:

InMemoryCollectionCaseRepository

Future:

SQL Server

Azure SQL

PostgreSQL

Idempotency Store

Current:

In-Memory Dictionary

Future:

Redis Cache

Distributed SQL Store

Event Publishing

Current:

FakeEventPublisher

Future:

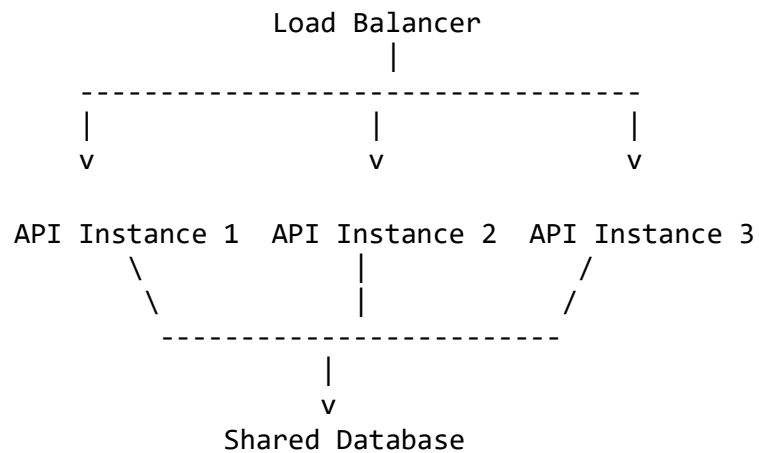
Azure Service Bus

Kafka

RabbitMQ

26.3 Horizontal Scaling

Target Architecture:



Benefits:

High Availability

Fault Tolerance

Scalability

27. Performance Considerations

27.1 Current Performance

The current implementation provides:

$O(1)$ Repository Lookup

$O(1)$ Idempotency Lookup

using in-memory collections.

27.2 Future Optimizations

Potential improvements:

Caching

Database Indexing

Connection Pooling

Asynchronous Messaging

27.3 API Response Times

Expected Targets:

Create Case < 100 ms

Get Case < 50 ms

Create PTP < 150 ms

These values are achievable with proper persistence and infrastructure.

28. Observability & Monitoring

28.1 Overview

Observability is critical in enterprise systems.

The architecture is designed to support:

Logging

Metrics

Tracing

Alerting

28.2 Logging

Recommended Logging Frameworks:

Serilog

NLog

Microsoft.Extensions.Logging

Example Events:

Case Created

Promise-To-Pay Created

Duplicate Request Rejected

Validation Failure

28.3 Metrics

Recommended Metrics:

Cases Created

PTPs Created

Duplicate Requests

API Response Times

Exception Counts

28.4 Distributed Tracing

Future implementation may include:

OpenTelemetry

Azure Monitor

Application Insights

Tracing Flow:

Request

|

v

API

|

v

Database

|

v

Message Bus

28.5 Health Checks

Recommended Endpoints:

GET /health

GET /ready

Purpose:

Kubernetes Probes

Load Balancer Checks

Operational Monitoring

29. Deployment Architecture

29.1 Current Deployment

Assessment Deployment:

Developer Machine



ASP.NET Core API

29.2 Target Production Architecture

Users



API Gateway



Load Balancer



Card Collections API



+----- SQL Database



+----- Redis Cache



+----- Service Bus

29.3 Containerization

Recommended:

Docker

Benefits:

Portability

Consistency

Cloud Deployment

29.4 CI/CD

Recommended Pipeline:

Build

|

v

Unit Tests

|

v

Static Analysis

|

v

Container Build

|

v

Deployment

Possible Platforms:

GitHub Actions

Azure DevOps

Jenkins

30. Risks & Mitigations

Risk 1

Duplicate Requests

Impact:

Data Integrity Issues

Mitigation:

Idempotency Store

Risk 2

Lost Audit Information

Impact:

Compliance Concerns

Mitigation:

AuditEntry Creation

Risk 3

Tight Coupling

Impact:

Reduced Maintainability

Mitigation:

Clean Architecture

Risk 4

Scaling Limitations

Impact:

Performance Degradation

Mitigation:

Distributed Storage

Horizontal Scaling

Caching

31. Future Enhancements

The architecture intentionally supports future growth.

Potential enhancements include:

Persistence Layer

Entity Framework Core

SQL Server

PostgreSQL

Distributed Caching

Redis

Event Driven Architecture

Azure Service Bus

Kafka

RabbitMQ

Authentication & Authorization

OAuth2

OpenID Connect

Azure AD

Payment Arrangement Support

Current:

One Promise-To-Pay Per Request

Future:

Installment Arrangements

Multi-Stage Payment Plans

Correlation Tracking

Support:

CorrelationId

Distributed Tracing

OpenTelemetry

Notifications

Future integrations:

SMS

Email

Push Notifications

32. Technology Stack

Framework

ASP.NET Core 10

Language

C# 14

Architecture

Clean Architecture

Domain Driven Design

Testing

MSTest

Documentation

Markdown

README

Architecture Design Document

API Documentation

Swagger / OpenAPI

33. Conclusion

The Card Collections Promise-To-Pay Service demonstrates a modern, maintainable, and extensible architecture aligned with enterprise software engineering practices.

The solution successfully delivers:

Collection Case Management

Promise-To-Pay Processing

Idempotent Request Handling

Audit Trail Recording

Domain Event Publication

Validation

Exception Handling

Unit Testing

The architecture prioritizes:

Maintainability

Testability

Scalability

Extensibility

Separation of Concerns

through the adoption of:

Clean Architecture

Domain Driven Design

Dependency Injection

RESTful API Design

While the current implementation uses in-memory components for simplicity, the design provides a clear migration path toward production-grade persistence, distributed caching, event-driven messaging, and cloud-native deployment models.

The resulting solution serves as a strong foundation for a scalable enterprise collections platform and demonstrates architectural practices appropriate for modern financial services applications.

Appendix A – Repository Structure

card-collections-ptp-service

README.md

docs/

└─ Architecture-Design-v1.0.md

src/

└─ CardCollections.Api
└─ CardCollections.Application
└─ CardCollections.Domain
└─ CardCollections.Infrastructure

tests/

└─ CardCollections.UnitTests

Appendix B – Key Design Patterns

Clean Architecture

Repository Pattern

Dependency Injection

Aggregate Root

Domain Events

Idempotency Pattern

Middleware Pattern

Appendix C – Assessment Deliverables

Source Code

GitHub Repository

README

Architecture Design Document

Unit Tests

Swagger API

